

Ciclo de Vida en JSF



Ciclo de Vida en JSF: Introducción y Ejemplo Práctico

El **ciclo de vida de JSF** es fundamental para entender cómo las solicitudes y respuestas son procesadas en una aplicación web basada en este framework. Este ciclo incluye una serie de fases predefinidas que permiten la transformación de una solicitud en una respuesta HTML.

En esta lección exploraremos las fases del ciclo de vida y presentaremos un ejemplo práctico con código, implementando un `PhaseListener` para monitorear el flujo y mostrar cómo manejar un caso básico con JSF.

Fases del Ciclo de Vida en JSF

1. **Restore View (Restaurar Vista):**
Esta fase se encarga de crear o restaurar la vista de la página solicitada.
2. **Apply Request Values (Aplicar Valores de Solicitud):**
Se obtienen los valores ingresados por el usuario y se asignan a los componentes correspondientes.
3. **Process Validations (Procesar Validaciones):**
JSF valida los datos basándose en las restricciones definidas (como longitud, formato, etc.).
4. **Update Model Values (Actualizar Modelo):**
Los datos validados se transfieren al modelo de la aplicación (beans administrados).
5. **Invoke Application (Invocar Aplicación):**
Aquí se ejecuta la lógica de negocio de la aplicación.

6. Render Response (Renderizar Respuesta):

Se genera la respuesta final en formato HTML para el cliente.

Ejemplo Práctico

En este ejemplo, configuraremos un listener para registrar las fases del ciclo de vida y crearemos una funcionalidad básica donde un usuario ingresa un nombre y se le redirige a una página de éxito o fallo dependiendo del valor ingresado.

Configuración de Dependencias

En el archivo `pom.xml`, añadimos las siguientes dependencias para incluir JSF (JakartaEE) y SLF4J para el manejo de logging:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>web</groupId>
    <artifactId>CicloVidaJSF</artifactId>
    <version>1.0</version>
    <packaging>war</packaging>
    <name>CicloVidaJSF</name>

    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <jakartaee>11.0.0-M1</jakartaee>
    </properties>

    <dependencies>
        <dependency>
            <groupId>jakarta.platform</groupId>
            <artifactId>jakarta.jakartaee-api</artifactId>
            <version>${jakartaee}</version>
            <scope>provided</scope>
        </dependency>
        <dependency>
            <groupId>org.slf4j</groupId>
            <artifactId>slf4j-api</artifactId>
            <version>2.0.16</version>
        </dependency>
        <dependency>
            <groupId>org.slf4j</groupId>
            <artifactId>slf4j-simple</artifactId>
```

```

        <version>2.0.16</version>
        <scope>runtime</scope>
    </dependency>

</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <version>3.12.1</version>
            <configuration>
                <source>21</source>
                <target>21</target>
            </configuration>
        </plugin>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-war-plugin</artifactId>
            <version>3.4.0</version>
        </plugin>
    </plugins>
</build>
</project>

```

Archivo log4j2.java

Agregamos el archivo log4j2.xml en la ruta src/main/resources:

```

<?xml version="1.0" encoding="UTF-8"?>
<Configuration status="INFO">
    <Appenders>
        <Console name="Console" target="SYSTEM_OUT">
            <PatternLayout pattern="%d{HH:mm:ss} [%t] %-5level %logger{36} -%msg%n"></PatternLayout>
        </Console>
    </Appenders>
    <Loggers>
        <Root level="info">
            <AppenderRef ref="Console" />
        </Root>
    </Loggers>
</Configuration>

```

Configuración de faces-config.xml

Este archivo configura el listener para registrar las fases del ciclo de vida en la ruta: Web Pages/WEB-INF (Pueden usar la ayuda de Netbeans para crear este archivo):

```
<?xml version='1.0' encoding='UTF-8'?>
<faces-config version="4.1"
    xmlns="https://jakarta.ee/xml/ns/jakartaee"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/web-facesconfig_4_1.xsd">
    <lifecycle>
        <phase-listener>
            beans.lifecycle.DebugListener
        </phase-listener>
    </lifecycle>
</faces-config>
```

Implementación del Listener

El listener `DebugListener` monitorea y registra mensajes antes y después de cada fase del ciclo de vida:

```
package beans.lifecycle;

import jakarta.faces.event.PhaseListener;
import jakarta.faces.event.PhaseEvent;
import jakarta.faces.event.PhaseId;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class DebugListener implements PhaseListener {

    private static final Logger log = LoggerFactory.getLogger(DebugListener.class);

    @Override
    public void beforePhase(PhaseEvent event) {
        log.info("Antes de la fase: " + event.getPhaseId());
    }

    @Override
    public void afterPhase(PhaseEvent event) {
        log.info("Despues de la fase: " + event.getPhaseId());
    }

    @Override
```

```
    public PhaseId getPhaseId() {  
        return PhaseId.ANY_PHASE;  
    }  
}
```

Bean de Modelo: Candidato

El bean `Candidato` representa el modelo de datos con una propiedad `nombre`:

```
package beans.model;  
  
import jakarta.enterprise.context.SessionScoped;  
import jakarta.inject.Named;  
import java.io.Serializable;  
import org.slf4j.Logger;  
import org.slf4j.LoggerFactory;  
  
@Named("candidato")  
@SessionScoped  
public class Candidato implements Serializable{  
  
    private static final Logger log = LoggerFactory.getLogger(Candidato.class);  
    private String nombre = "Introduce tu nombre";  
  
    public String getNombre() {  
        return nombre;  
    }  
  
    public void setNombre(String nombre) {  
        log.info("Actualizando nombre a: " + nombre);  
        this.nombre = nombre;  
    }  
}
```

Bean de Respaldo: vacanteForm

El bean `VacanteForm` contiene la lógica de negocio para procesar el formulario:

```
package beans.backing;  
  
import beans.model.Candidato;  
import jakarta.enterprise.context.RequestScoped;  
import jakarta.inject.Inject;  
import jakarta.inject.Named;
```

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@Named("vacanteForm")
@RequestScoped
public class VacanteForm {

    private static final Logger log = LoggerFactory.getLogger(VacanteForm.class);

    @Inject
    private Candidato candidato;

    public void setCandidato(Candidato candidato){
        this.candidato = candidato;
    }

    public String enviar(){
        log.info("Procesando solicitud para: " + this.candidato.getNombre());
        if(this.candidato.getNombre().equals("Juan")){
            return "exito?faces-redirect=true";//exito.xhtml
        }
        else{
            return "fallo?faces-redirect=true";//fallo.xhtml
        }
    }
}
```

Explicación del Ejemplo

1. **Registro del Ciclo de Vida:**
El `PhaseListener` permite observar las fases del ciclo de vida y registrar mensajes antes y después de cada una.
2. **Validación de Usuario:**
El bean `VacanteForm` verifica el nombre ingresado. Si es "Juan", redirige a la página de éxito; en caso contrario, a la de fallo.
3. **Modularidad:**
Separar el modelo (`Candidato`), la lógica de negocio (`VacanteForm`) y la configuración (`faces-config.xml`) mejora la mantenibilidad y claridad.

Conclusión

Este ejemplo muestra cómo manejar el ciclo de vida de JSF mientras se implementan funcionalidades prácticas. El uso de un `PhaseListener` no solo facilita la depuración, sino

que también proporciona una visión más clara del flujo de ejecución. Esta estructura modular y bien organizada es ideal para aplicaciones JSF escalables.

Saludos!

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)